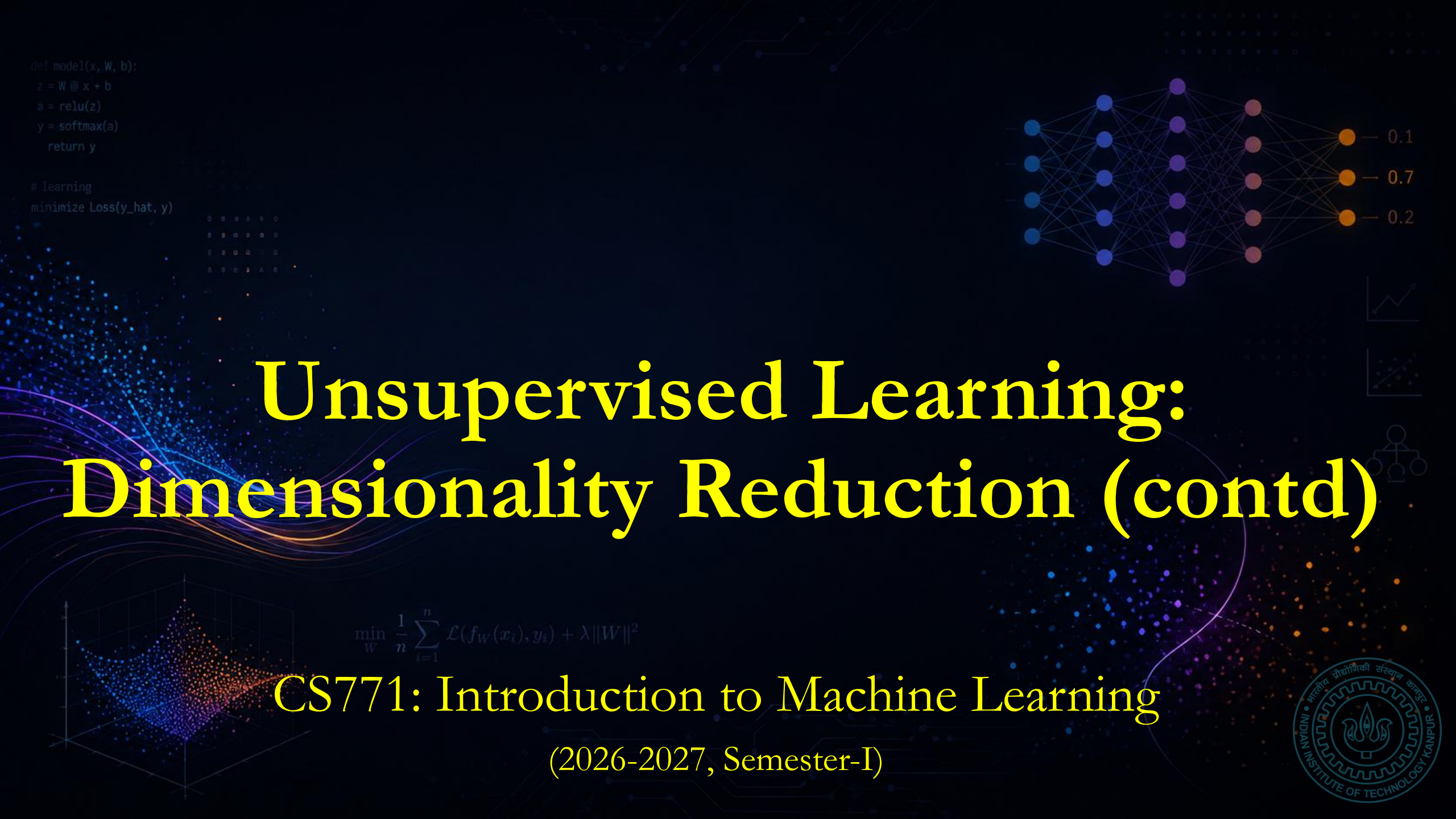


Unsupervised Learning: Dimensionality Reduction (contd)

CS771: Introduction to Machine Learning

(2026-2027, Semester-I)

Recap: PCA

Total variance of projected data (assuming inputs are already centered)

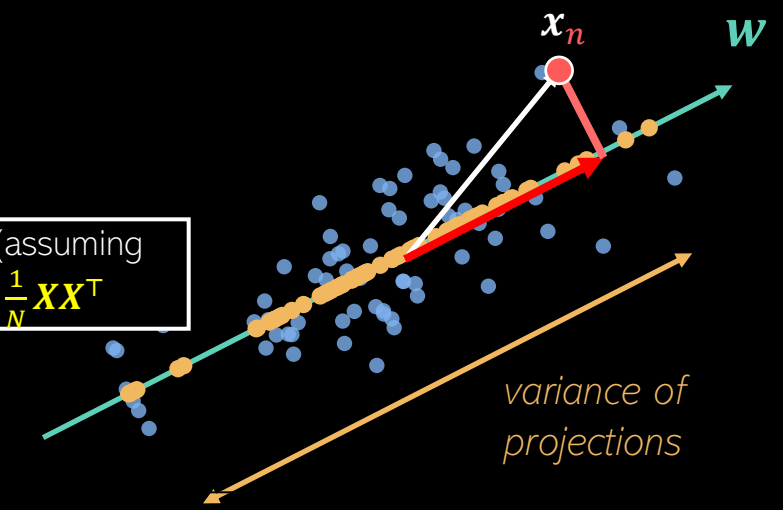
Maximize the variance of the projections

$$\max_{\|\boldsymbol{w}\|=1} \frac{1}{N} \sum_{n=1}^{N} \left(\boldsymbol{w}^\top \boldsymbol{x}_n\right)^2 = \max_{\|\boldsymbol{w}\|=1} \boldsymbol{w}^\top \boldsymbol{S} \boldsymbol{w}$$

Projection along $\boldsymbol{w}$

The $D \times D$ cov matrix of inputs (assuming each input is $\boldsymbol{x}_n$ centered): $\boldsymbol{S} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top$

Question: why do we need this constraint in this optimization probem?



$$= \min_{\boldsymbol{w}} \left[-\boldsymbol{w}^\top \boldsymbol{S} \boldsymbol{w}\right] \quad \text{s.t. } \boldsymbol{w}^\top \boldsymbol{w} = 1$$

Constrained optimization with an equality constraint. Lagrangian:

$$\min_{\boldsymbol{w}} \max_{\lambda} \left[-\boldsymbol{w}^\top \boldsymbol{S} \boldsymbol{w} + \lambda(\boldsymbol{w}^\top \boldsymbol{w} - \mathbf{1})\right] \Rightarrow \boxed{\boldsymbol{S}\boldsymbol{w} = \lambda \boldsymbol{w}}$$

$\boldsymbol{w}$ is the top eigenvector of $\boldsymbol{S}$

Direction of largest variance in the data

Eigendec of $\boldsymbol{S}$ gives us D such orthonormal directions $\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_D$

Respective variances (largest to smallest): $\lambda_1, \lambda_2, \ldots, \lambda_D$

Recap: PCA

Projection of $\boldsymbol{x}_n$ along top K evecs $\boldsymbol{W}_K = [\boldsymbol{w}_1, \boldsymbol{w}_2, \ldots, \boldsymbol{w}_K] \in \mathbb{R}^{D \times K}$

PCA based projection/embedding of $\boldsymbol{x}_n$

Linear projection of $\boldsymbol{x}_n$ to the subspace defined by top K eigenvectors

$$\boldsymbol{z}_n = \begin{bmatrix} z_{n1} \\ z_{n2} \\ \vdots \\ z_{nK} \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_1^\top \boldsymbol{x}_n \\ \boldsymbol{w}_2^\top \boldsymbol{x}_n \\ \vdots \\ \boldsymbol{w}_K^\top \boldsymbol{x}_n \end{bmatrix} = \boldsymbol{W}_K^\top \boldsymbol{x}_n \in \mathbb{R}^K$$

$$\boxed{\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}_k}$$

$(N \times K) \quad (N \times D)(D \times K)$

Given $\boldsymbol{z}_n \in \mathbb{R}^K$ we can get the reconstruction of $\boldsymbol{x}_n$ as

$$\begin{aligned} \hat{\boldsymbol{x}}_n &= z_{n1}\boldsymbol{w}_1 + z_{n2}\boldsymbol{w}_2 + \cdots + z_{nK}\boldsymbol{w}_K \\ &= (\boldsymbol{w}_1^\top \boldsymbol{x}_n)\boldsymbol{w}_1 + (\boldsymbol{w}_2^\top \boldsymbol{x}_n)\boldsymbol{w}_2 + \cdots + (\boldsymbol{w}_K^\top \boldsymbol{x}_n)\boldsymbol{w}_K \\ &= \boldsymbol{W}_K \boldsymbol{z}_n \\ &= \boldsymbol{W}_K \boldsymbol{W}_K^\top \boldsymbol{x}_n \end{aligned}$$

$$\boxed{\hat{\boldsymbol{X}} = \boldsymbol{Z}\boldsymbol{W}_K^\top}$$

$(N \times D) \quad (N \times K)(K \times D)$

Zero if $K = D$

Reconstruction error: $\|\boldsymbol{x}_n - \hat{\boldsymbol{x}}_n\|^2 = \|\boldsymbol{x}_n - \boldsymbol{W}_K \boldsymbol{W}_K^\top \boldsymbol{x}_n\|^2$

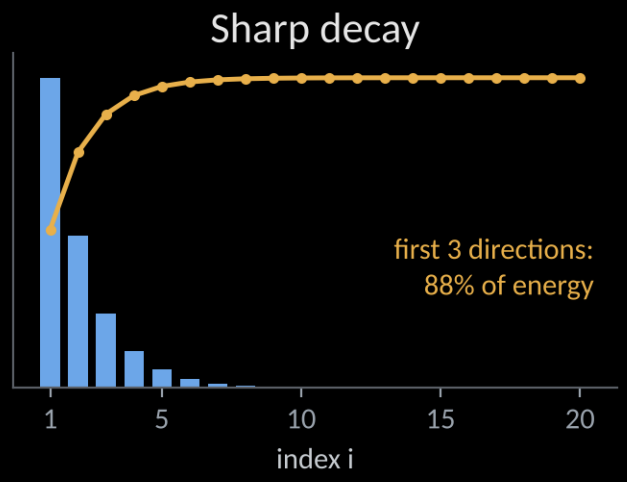
Eigenvals indicate how much we can throw away

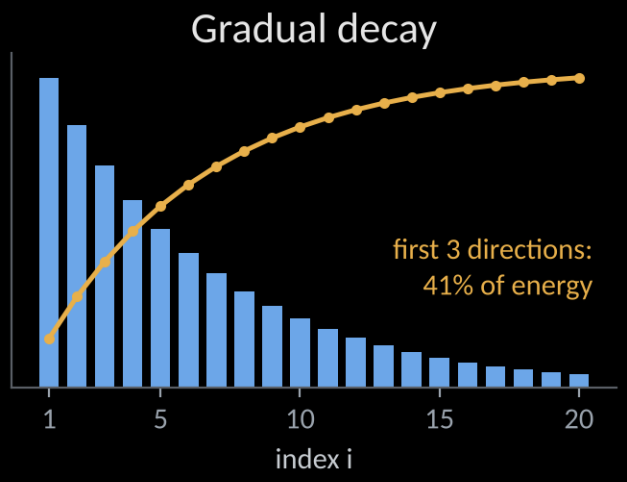
$$\text{Eigenvalue spectrum} = \{\lambda_1, \lambda_2, \dots, \lambda_D\}$$

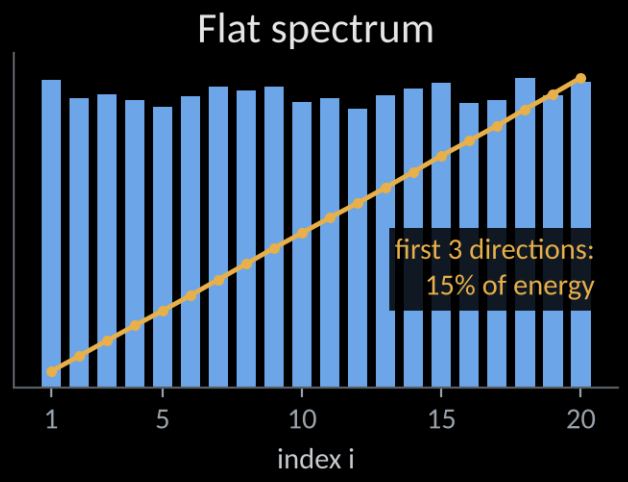
bars: eigenvalues λ_i (scaled)

$$\text{cumulative energy} = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{i=1}^{D} \lambda_i}$$

line: cumulative energy





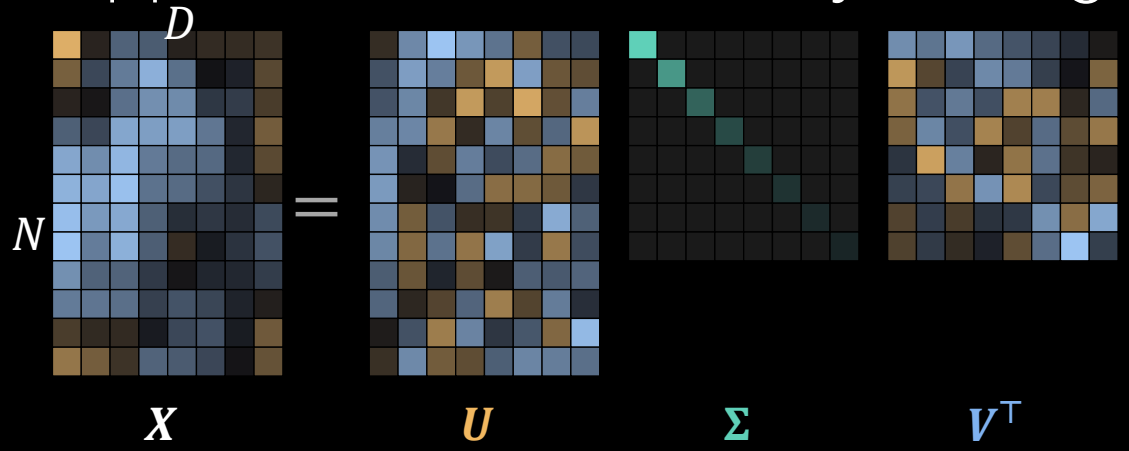


- Sharp decay: a few directions dominate.
- Gradual decay: moderate compression may be possible.
- Flat spectrum: no obvious small linear dim-red possible

Spectrum thus helps us choose an appropriate value of K

Singular Value Decomposition (SVD)

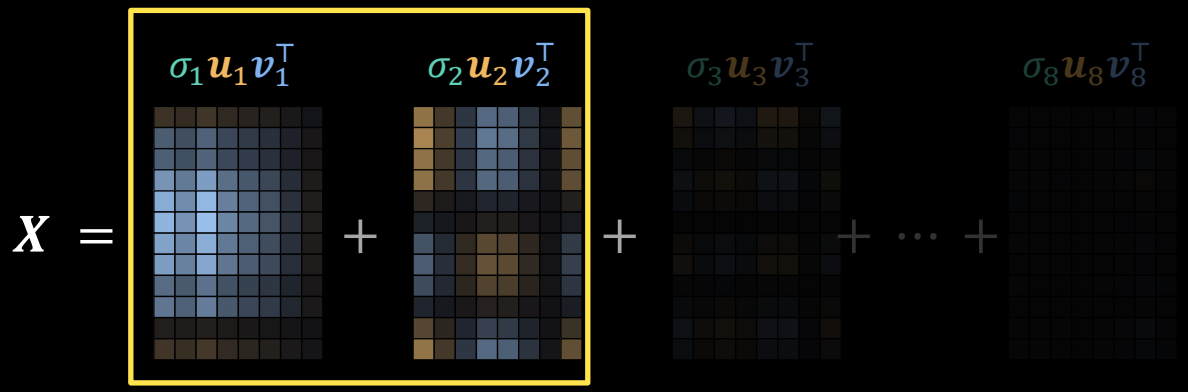
Approximate a matrix by a weighted sum of K rank-1 pieces



$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top = \sum_{k=1}^{r=\min\{N,D\}} \sigma_k \boldsymbol{u}_k \boldsymbol{v}_k^\top$$

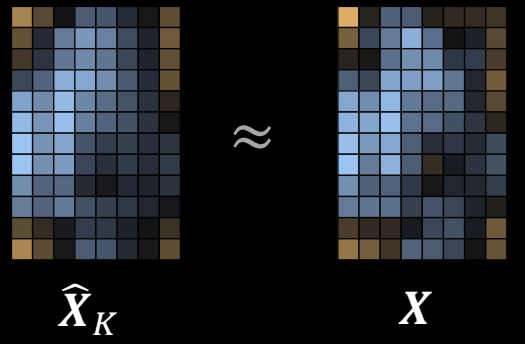
$\boldsymbol{U}, \boldsymbol{V}$: orthonormal columns (left / right singular vectors)

$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$: singular values (diagonal of $\boldsymbol{\Sigma}$)



later pieces carry less and less info about $\boldsymbol{X}$

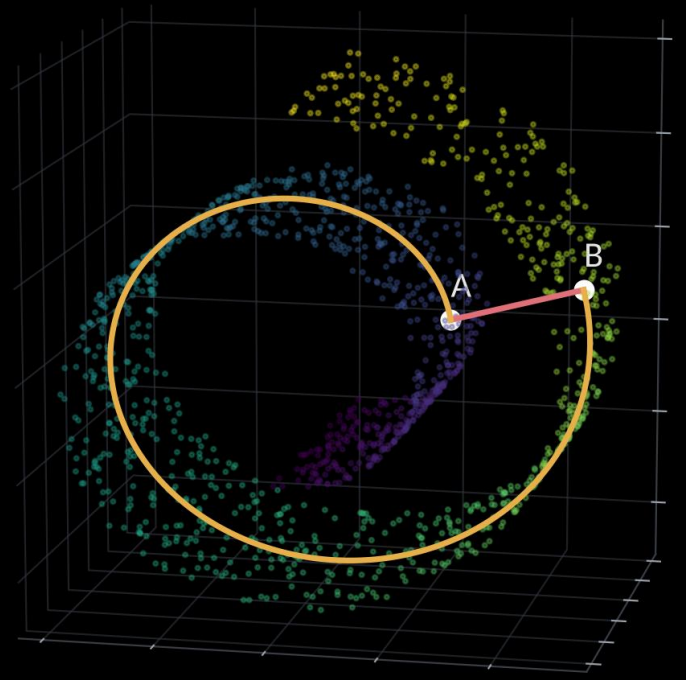
Keep the top $K = 2$ pieces:

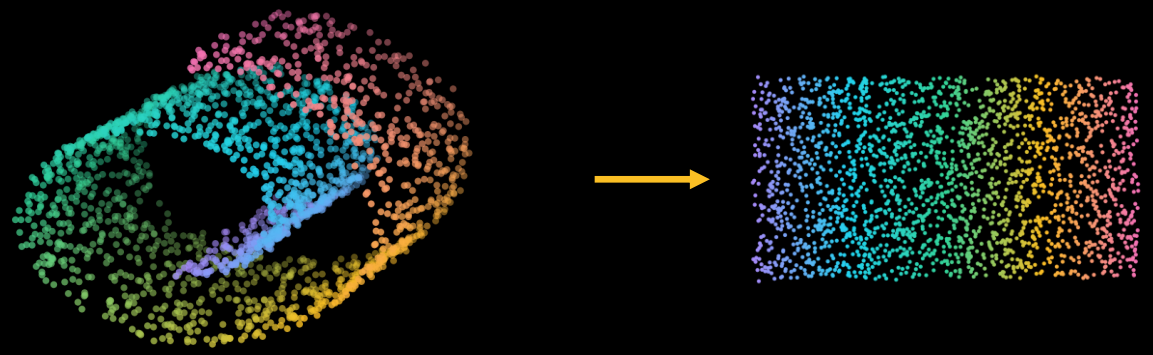


Best rank-K approximation: smallest $\|\boldsymbol{X} - \hat{\boldsymbol{X}}_K\|$ *(Eckart–Young)*

Link to PCA (centered $\boldsymbol{X}$): $\boldsymbol{V}_K$ (first K columns of $\boldsymbol{V}$) $= \boldsymbol{W}_K$ (in PCA) $\quad \boldsymbol{Z} = \boldsymbol{X}\boldsymbol{W}_K = \boldsymbol{U}_K\boldsymbol{\Sigma}_K$

When linear projections don't work





Extrinsic: 3-D (x, y, z)

Intrinsic: 2-D sheet

The Swiss roll: a flat 2-D sheet rolled up in 3-D space

Instead of preserving variance or reconstruction error, in some cases, preserving local geometry may be more appropriate

Focusing on preserving the local neighborhood may be a better idea

Generally two ways to do dimensionality reduction in such cases

(1) Nonlinearlize a linear DR method such as PCA

(2) Use local geometry preservation or manifold learning methods

Kernel PCA

Recall kernels: $k(\boldsymbol{x}_n, \boldsymbol{x}_m) = \phi(\boldsymbol{x}_n)^\top \phi(\boldsymbol{x}_m)$

Fact: each eigenvector lies in the data span: $\boldsymbol{w}_k = \sum_{n=1}^{N} \alpha_{nk}\, \boldsymbol{\phi}(\boldsymbol{x}_n)$

The $D \times D$ eigenproblem $\boldsymbol{S}\boldsymbol{w}_k = \boldsymbol{\lambda}\boldsymbol{w}_k$ becomes an $N \times N$ problem

$\boldsymbol{K}$ is the $N \times N$ kernel matrix

$$\tilde{\boldsymbol{K}}\boldsymbol{\alpha}_k = \lambda_k \boldsymbol{\alpha}_k, \quad \tilde{\boldsymbol{K}} = \boldsymbol{H}\boldsymbol{K}\boldsymbol{H}, \quad \boldsymbol{H} = \boldsymbol{I} - \frac{1}{N}\boldsymbol{1}\boldsymbol{1}^\top$$

$\boldsymbol{H}$ centers the data in $\boldsymbol{\phi}$ feature space

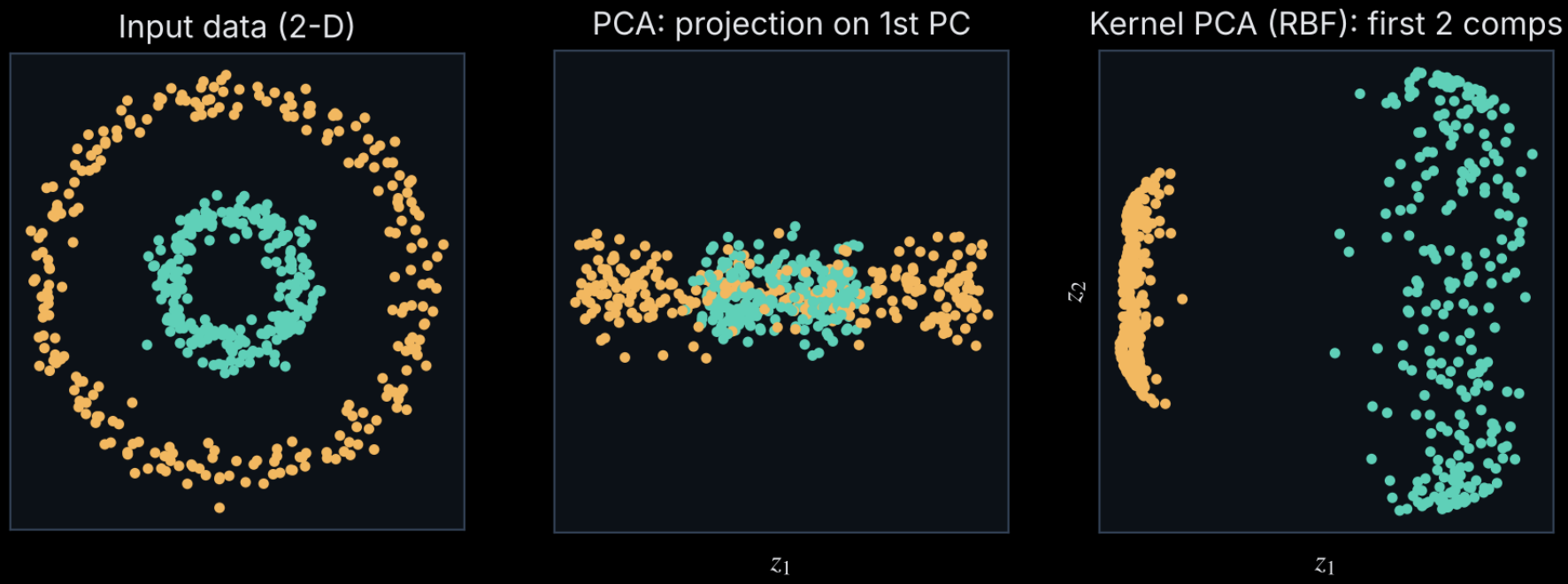
Embedding of any input $\boldsymbol{x}$ (training or new)

$$z_k(\boldsymbol{x}) = \boldsymbol{w}_k^\top \boldsymbol{\phi}(\boldsymbol{x}) = \sum_{n=1}^{N} \alpha_{nk}\ \tilde{k}(\boldsymbol{x}_n, \boldsymbol{x}) \quad k = 1,2,\ldots,K$$

With a linear kernel, KPCA is equivalent to linear PCA (but still advantageous if $N < D$ because eigenproblem is computationally easier)



Kernel PCA in Action



- RBF kernel $k(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\gamma \parallel \boldsymbol{x} - \boldsymbol{x}' \parallel^2\right)$: the rings become linearly separable
- Cost: $N \times N$ kernel matrix, $O\left(N^3\right)$ eigendecomposition
- Scalable approximations: Nyström, random Fourier features (approximate $\boldsymbol{\phi}$, then linear PCA)
- Kernel choice and hyperparameters (e.g., γ) matter

Multi-dimensional Scaling (MDS)

Not necessarily Euclidean distances

Sometimes instead of inputs we only their have pairwise distances d_{nm}

Goal: find $\boldsymbol{z}_n \in \mathbb{R}^K$ with $\|\boldsymbol{z}_n - \boldsymbol{z}_m\| \approx d_{nm}$

$\boldsymbol{D}^{(2)}$: squared distances d_{nm}^2; $\boldsymbol{H} = \boldsymbol{I} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$ does centering

MDS does eigendec of this $N \times N$ matrix: $\boldsymbol{B} = -\frac{1}{2}\boldsymbol{H}\boldsymbol{D}^{(2)}\boldsymbol{H}$

Embeddings: $\boldsymbol{Z} = \boldsymbol{U}_K \boldsymbol{\Lambda}_K^{1/2}$ ($\boldsymbol{U}_K$, $\boldsymbol{\Lambda}_K$: top-K evec/eval pairs of $\boldsymbol{B}$)

Input: only pairwise distances (km)
(6 of the 16 cities shown)

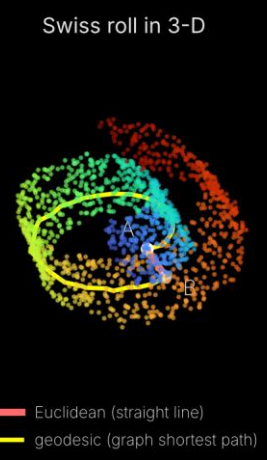
	Del	Mum	Kol	Che	Kan	Guw
Delhi	0	1147	1303	1756	390	1460
Mumbai	1147	0	1654	1033	1120	2086
Kolkata	1303	1654	0	1358	920	524
Chennai	1756	1033	1358	0	1487	1882
Kanpur	390	1120	920	1487	0	1138
Guwahati	1460	2086	524	1882	1138	0

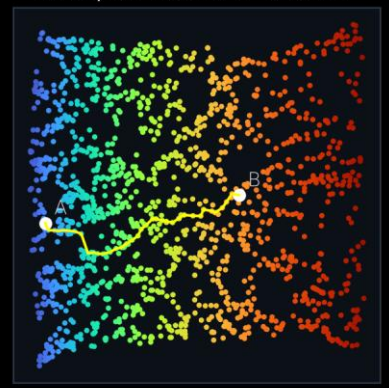
Output of classical MDS ($K = 2$): a map of India!
(rotated/reflected to match the usual map orientation)

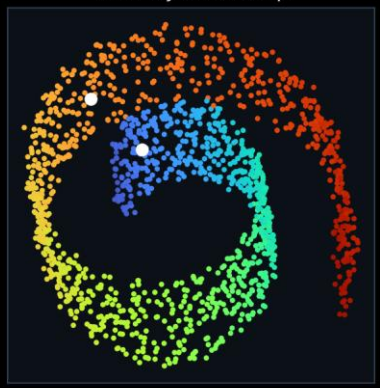


16 cities, great-circle distances (km) → a 2-D map. Top-2 eigenvalues hold 99.9% of the spectrum

MDS on "Geodesic Distances" = Isomap algorithm







Stochastic Neighbor Embedding (and Variants)

SNE: Preserve neighborhoods probabilistically (Hinton & Roweis, 2002)

Probability that point i picks point j as its neighbor in the original space

$$p_{j|i} = \frac{\exp\left(-\| \boldsymbol{x}_i - \boldsymbol{x}_j \|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\| \boldsymbol{x}_i - \boldsymbol{x}_k \|^2 / 2\sigma_i^2\right)}$$

Probability that point i picks point j as its neighbor in the projected lower-dim space

$$q_{j|i} = \frac{\exp\left(-\| \boldsymbol{z}_i - \boldsymbol{z}_j \|^2\right)}{\sum_{k \neq i} \exp\left(-\| \boldsymbol{z}_i - \boldsymbol{z}_k \|^2\right)}$$

Learn $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N$ by gradient descent so that the q's match the p's:

$$\mathcal{L}(\boldsymbol{Z}) = \sum_i \mathrm{KL}\left(P_i \parallel Q_i\right) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

Uniform Manifold Approximation and Projection

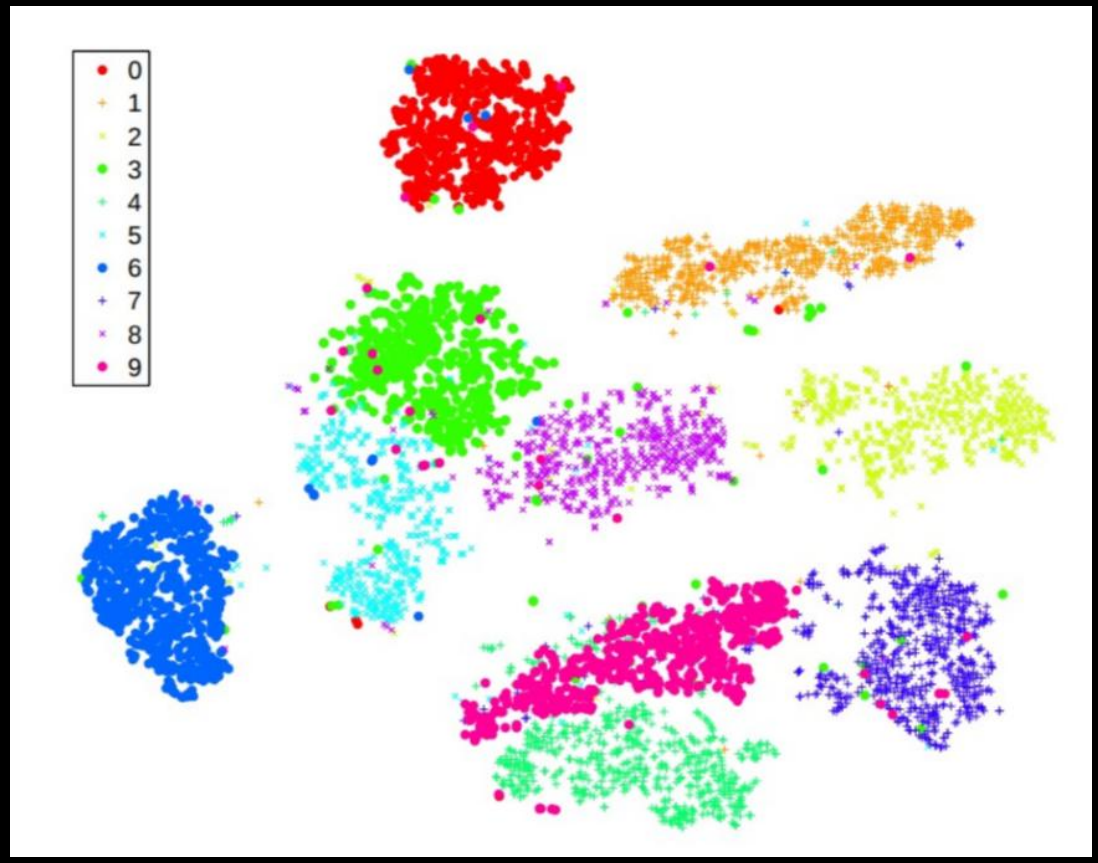
t-SNE and UMAP also based on preserving neighborhood like SNE

- t-SNE reduces the "crowding problem" of SNE: clearer low-dimensional cluster separation
- UMAP is usually faster and preserve more global structure in the data



SNE and variants

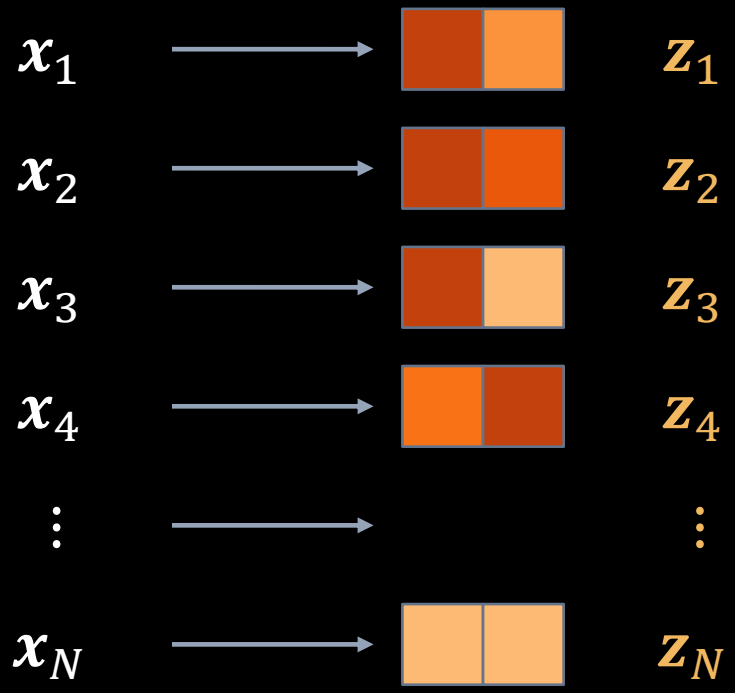
Especially useful for visualizing data by projecting to 2D or 3D



Result of visualizing MNIST digits data in 2D (Figure from van der Maaten and Hinton, 2008)

Parametric t-SNE/UMAP

"Non-parametric" (standard t-SNE/UMAP)



a lookup table: $N \times K$ free parameters

new point x_*? must re-optimize all embeddings including those of training data. Slow and expensive!

Parametric



same t-SNE/UMAP loss, but on $z_n = f_\theta(x_n)$

Don't estimate the z_n's. Estimate f_θ itself

Minimize loss w.r.t. θ

new point: $z_* = f_\theta(x_*)$, a single forward pass

Random Projections and the JL Lemma

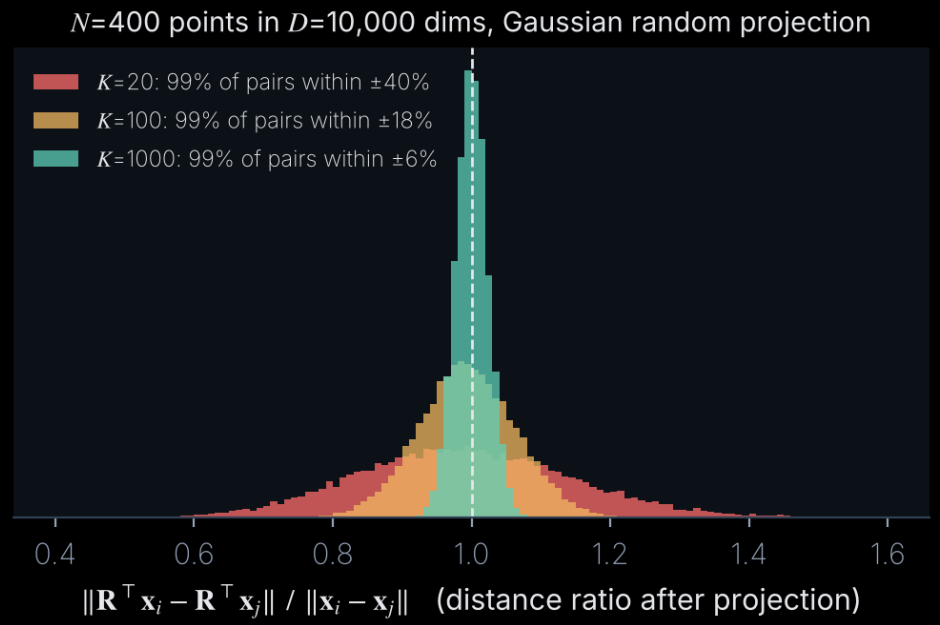
No training: $\boldsymbol{z}_n = \boldsymbol{R}^\top \boldsymbol{x}_n$, entries of $\boldsymbol{R} \in \mathbb{R}^{D \times K}$ i.i.d. from $\mathcal{N}(0, 1/K)$

Johnson-Lindenstrauss lemma: for any N points and $0 < \epsilon < 1$, if $K = O(\log N/\epsilon^2)$, then with high probability

Pairwise Euclidean distances preserved after projection

Johnson Lindenstraus (JL) Lemma

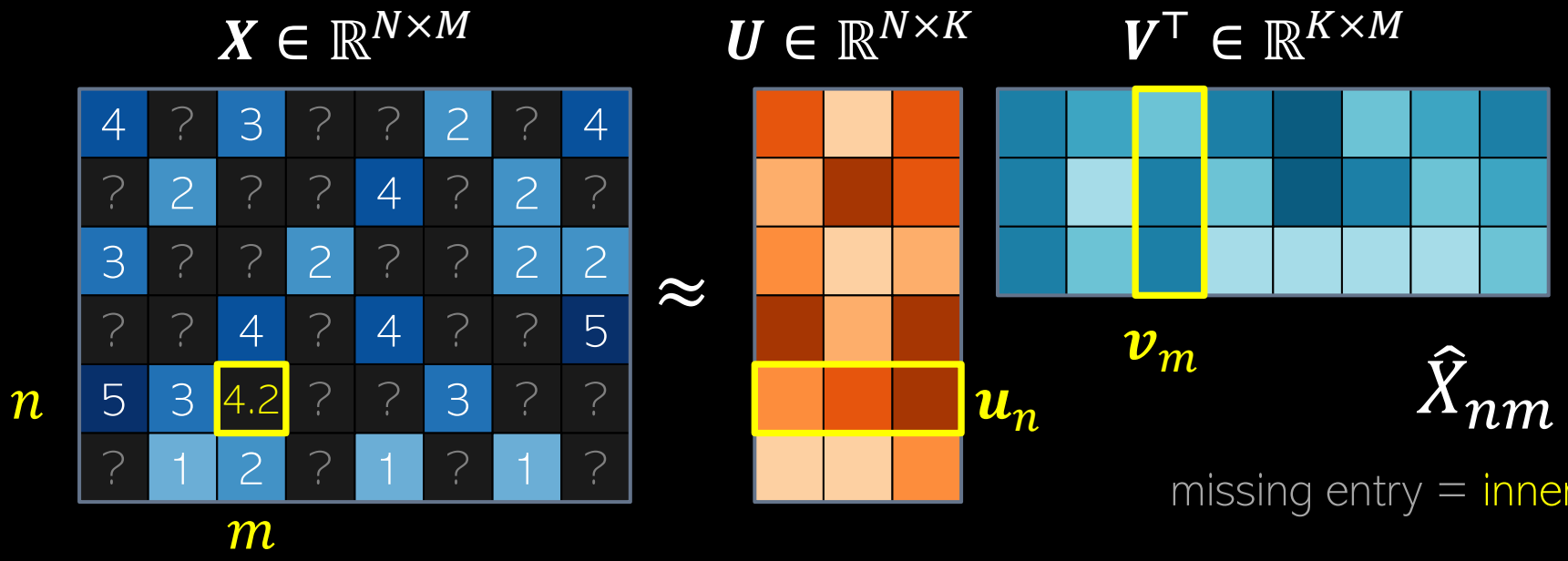
$$(1-\epsilon)\,\|\,\boldsymbol{x}_n - \boldsymbol{x}_m\,\|^2 \leq \|\,\boldsymbol{z}_n - \boldsymbol{z}_m\,\|^2 \leq (1+\epsilon)\,\|\,\boldsymbol{x}_n - \boldsymbol{x}_m\,\|^2 \quad \forall\, n, m$$



- Using sparse $(\pm 1/0)$ projections for $\boldsymbol{R}$ is even cheaper
- Uses: sketching, locality-sensitive hashing for nearest-neighbor search, speeding up other algorithms
- Unlike PCA, ignores the data, so it needs a larger K for the same error

Embeddings via Matrix Factorization

E.g., N users (type 1) rate M movies (type 2); ? = unrated



$$\hat{X}_{nm} = \boldsymbol{u}_n{}^\top \boldsymbol{v}_m$$

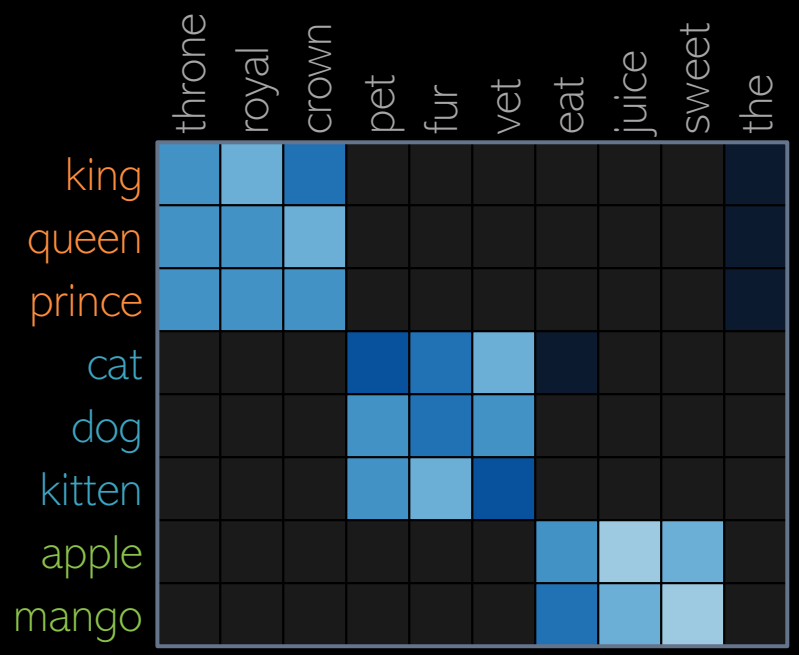
missing entry = inner product of embeddings

$$\min_{U,V} \sum_{(n,m)\in\Omega} (X_{nm} - \boldsymbol{u}_n{}^\top \boldsymbol{v}_m)^2 + \lambda\left(\sum_n \|\boldsymbol{u}_n\|^2 + \sum_m \|\boldsymbol{v}_m\|^2\right)$$

Sum over observed entries Ω only

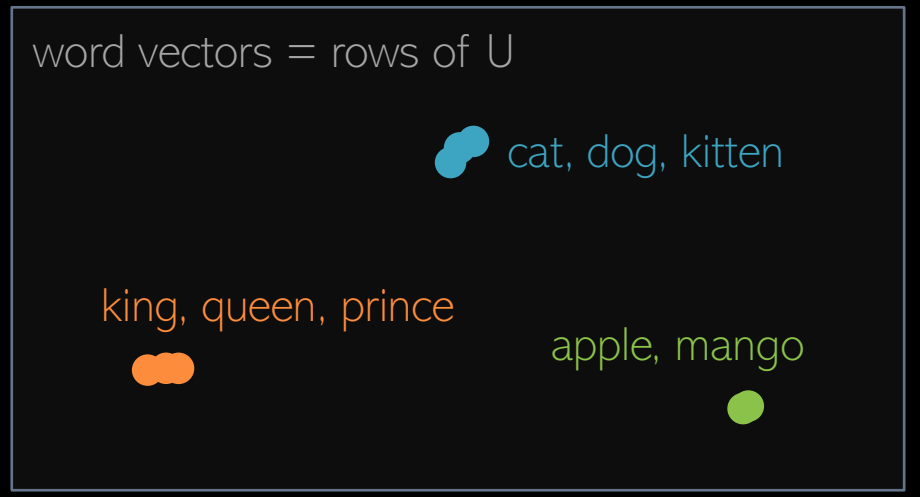
Keeps embeddings small

Word2vec/Glove as Matrix Factorization



$\mathrm{PMI}(w, c) = \log \frac{P(w,c)}{P(w)P(c)} > 0$:
co-occur more than chance

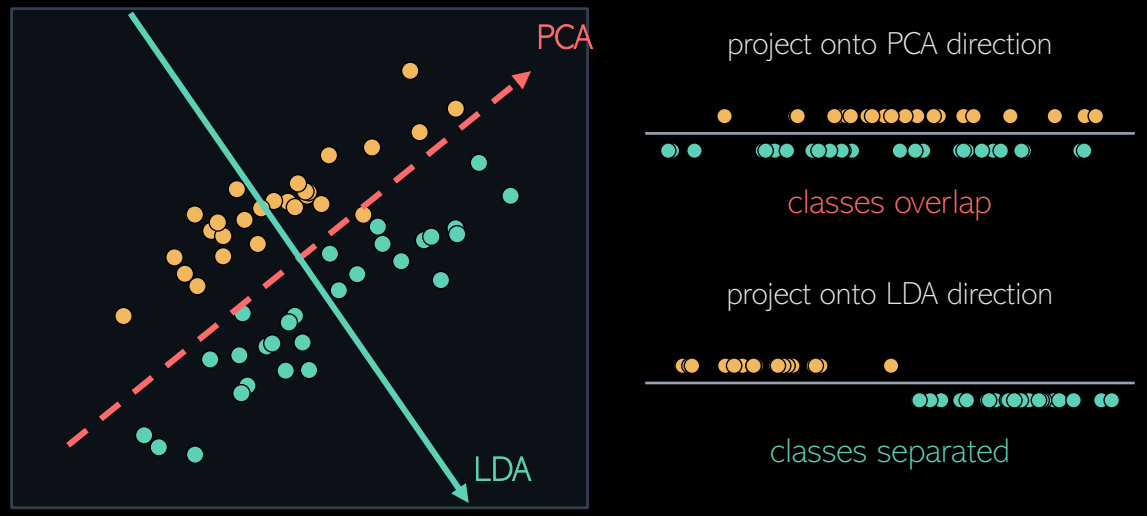
$$X \approx UV^{\top}$$
SVD



- word2vec: $\boldsymbol{u}_w{}^{\top}\boldsymbol{v}_c = \mathrm{PMI}(w, c) - \log k$ (Levy & Goldberg (2014))
- GloVe: $\boldsymbol{u}_w{}^{\top}\boldsymbol{v}_c + b_w + b_c \approx \log X_{wc}$ (counts)

Task-Oriented Dimensionality Reduction

A classical example: Linear Discriminant Analysis

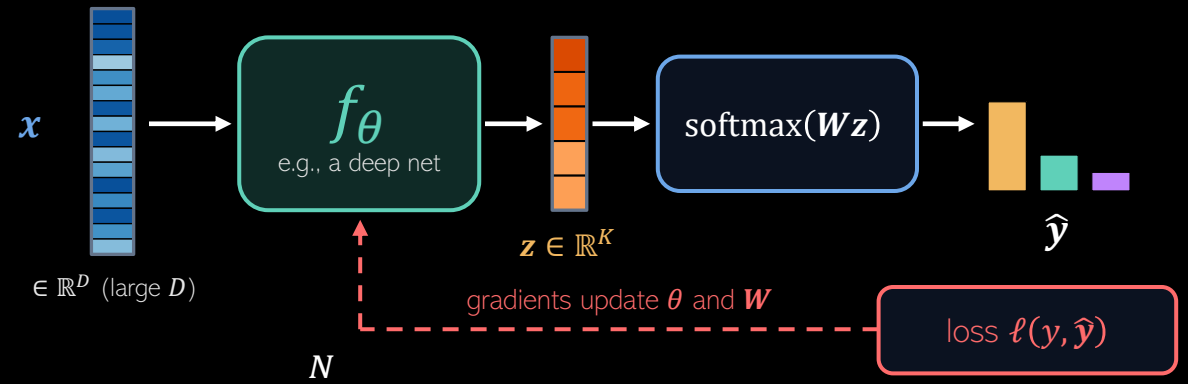


Fisher: maximize separation of class means relative to within-class spread

$$\max_{\boldsymbol{w}} \frac{\left(\boldsymbol{w}^\top(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\right)^2}{\boldsymbol{w}^\top \boldsymbol{S}_W \boldsymbol{w}} \Rightarrow \boldsymbol{w} \propto \boldsymbol{S}_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$

C classes: top eigenvectors of $\boldsymbol{S}_W^{-1}\boldsymbol{S}_B$ (within/between-class covariance), at most $C-1$ dims

More generally: learn the embedding via a task loss



$$\min_{\theta, \boldsymbol{W}} \sum_{n=1}^{N} \ell\left(y_n,\ \text{softmax}\left(\boldsymbol{W} f_\theta(\boldsymbol{x}_n)\right)\right)$$



- No variance or reconstruction objective: $\boldsymbol{z}$ keeps only what helps the task
- Classes end up (nearly) linearly separable in $\boldsymbol{z}$-space
- $\boldsymbol{z}$ is reusable: retrieval, clustering, other tasks

Unsupervised dim-red preserves properties of $\boldsymbol{X}$ (variance, distances, neighbors). Task-oriented dim-red preserves what predicts $\boldsymbol{y}$: LDA = linear f + class-separation objective; deep nets = nonlinear f_θ trained end-to-end on the task loss

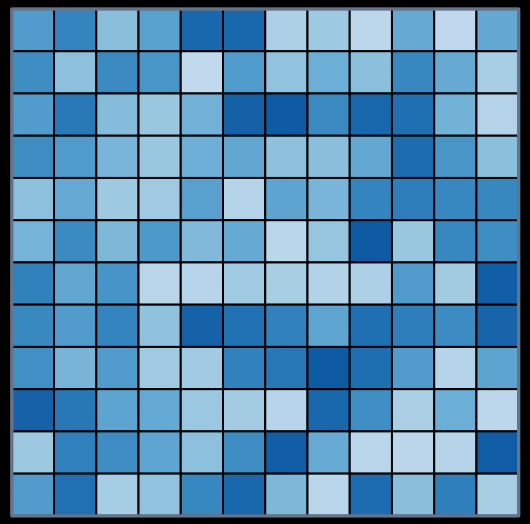
Dim-Red of Parameters: Low-Rank Adaptation

A large weight matrix can often be approximated by low-rank matrix

$$W = UV^{\top}$$

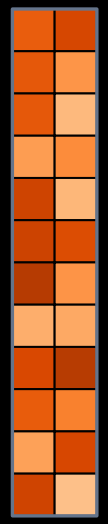
Adapting a large pretrained model: learn an update $W_0 \rightarrow W_0 + \Delta W$ to a pre-trained (frozen) weight matrix $W_0 \in \mathbb{R}^{D_{in} \times D_{out}}$

$W_0 \in \mathbb{R}^{D_{in} \times D_{out}}$ (frozen)



+

$B \in \mathbb{R}^{D_{in} \times r}$





$A \in \mathbb{R}^{r \times D_{out}}$

Low-Rank Adaptation - LoRA (Hu et al., 2021): restrict ΔW to rank $r \ll \min(D_{in}, D_{out})$, by assuming it to be defined as a low-rank matrix

$$\Delta W = BA$$

rank $\leq r$

$$W = W_0 + BA$$

only A, B are learned

Example: $D_{in} = D_{out} = 4096$, $r = 8$: $D_{in} \times D_{out} \approx$ 16.8M vs. $r(D_{in} + D_{out}) =$ 65,536 trainable numbers per matrix (256× fewer)